



Universidade do Minho
Escola de Engenharia

1º Ano Mestrado em Engenharia Biomédica
Informática Médica

Ano Letivo 2025/2026

Trabalho Prático I

Processamento de Linguagem Natural

pg59861 Bárbara Grandinho
pg59892 Laura Ferreira

15/04/2026

Índice

1	Introdução	3
2	Documentos Processados	4
3	Arquitetura	5
4	Tratamento de Dados	6
4.1	<i>glossario_ministerio_saude.pdf</i>	6
4.2	<i>glossario_neologismos.pdf</i>	10
4.3	<i>wipopearl.pdf</i>	14
4.4	<i>medicina.pdf</i>	16
4.5	Extração e Unificação Transversal de Siglas e Abreviaturas	18
5	Modelação do Dicionário Final (JSON)	20
6	Desafios e Limitações Encontradas	22
7	Conclusão	24

1. Introdução

Este relatório tem como principal objetivo descrever a metodologia e os resultados do primeiro trabalho prático realizado na Unidade Curricular de Processamento de Linguagem Natural em Engenharia Biomédica. De forma sucinta, o projeto consistiu na extração automatizada de informação relevante a partir de quatro documentos médicos, fornecidos em formato PDF, e na sua posterior estruturação e preservação em formato JSON.

Para manipular os textos, identificar padrões e isolar os conceitos médicos das suas respectivas definições, categorias e traduções, recorreu-se à utilização de Expressões Regulares (*Regex*) em ambiente Python. Esta abordagem permitiu efetuar operações complexas de pesquisa, substituição e extração sobre grandes volumes de texto não estruturado. Adicionalmente, as expressões regulares foram fundamentais para a fase de data cleaning (limpeza de dados), garantindo a remoção de elementos ruidosos inerentes à conversão de PDF para formato markup (XML), tais como cabeçalhos, rodapés, números de página, caracteres especiais (ligaduras tipográficas) e índices remissivos.

2. Documentos Processados

Para a realização deste projeto e construção da base de dados terminológica final, procedeu-se à extração de informação dos seguintes quatro documentos:

- ***glossario_ministerio_saude.pdf***: Um documento técnico abrangente que compila termos de saúde pública, contendo a designação do conceito, a sua área temática (categoria) e a respetiva definição clínica ou administrativa.
- ***wipopearl.pdf***: Um glossário de cariz multilingue, focado na terminologia associada à área da saúde, que exige a extração não só do termo e definição em língua inglesa, mas também o mapeamento das suas traduções em diversos outros idiomas.
- ***glossario_neologismos.pdf***: Um documento focado em novos termos médicos (neologismos), exigindo a extração de conceitos recentemente adotados na prática clínica.
- ***medicina.pdf***: Um glossário com definições e conceitos gerais da prática médica que complementa os restantes dicionários, enriquecendo o ficheiro final com vocabulário de anatomia e patologia geral.

A constituição deste conjunto documental guiou-se por um duplo critério de seleção: a complementaridade temática e a diversidade de desafios técnicos. Do ponto de vista temático, as obras escolhidas convergem para a criação de uma base de dados unificada altamente abrangente no domínio da saúde. A intersecção de terminologias de saúde pública e administração hospitalar com patologia geral, acrescida de vocabulário clínico emergente (neologismos) e enriquecida pelo mapeamento multilingue, assegurou a construção de um glossário transversal e semanticamente rico.

Em paralelo, sob a ótica de engenharia de dados, a acentuada heterogeneidade estrutural destes ficheiros providenciou o cenário de teste ideal. A necessidade de processar documentos com diferentes níveis de ruído tipográfico, oclusões de texto, âncoras morfológicas distintas e estruturas descentralizadas (como siglas embutidas), exigiu a conceção de uma arquitetura de extração verdadeiramente modular. Esta escolha forçou o sistema a ir além da extração linear, provando a robustez e a escalabilidade dos algoritmos perante diferentes paradigmas de formatação de PDFs.

3. Arquitetura

O objetivo final deste trabalho prático consistiu na obtenção de um documento estruturado (JSON Geral) que reunisse todo o vocabulário médico processado de forma limpa, permitindo a sua fácil consulta computacional.

A arquitetura da solução foi dividida nas seguintes etapas sequenciais:

- **Conversão de Formato:** Todos os documentos originais em formato .pdf foram inicialmente convertidos para o formato .xml através da ferramenta de linha de comandos `pdftohtml -xml`.
- **Processamento Individual** (Módulos): Para cada um dos quatro ficheiros XML gerados, foi desenvolvido um script em Python individual. Cada script efetuou a limpeza de metadados das *tags* XML (como `<page>`, `<fontspec>`, `<i>`, `<b>`) e utilizou regras de *Regex* específicas à formatação visual de cada livro para capturar os Termos, Categorias, Traduções e Descrições.
- **Armazenamento Temporário:** Cada script gerou um dicionário em Python que foi exportado como um ficheiro JSON individual e temporário (e.g., `ministerio_sau_de_temp.json`).
- **Agregação** (*Merge*): Numa fase final, foi construído um script agregador que importou os quatro ficheiros JSON individuais e fundiu os dados, evitando sobreposições e garantindo que, se um termo existisse em mais do que um documento, as suas informações (como descrições alternativas ou traduções) fossem combinadas numa única entrada. O ficheiro final foi estruturado de forma a privilegiar a consulta rápida, posicionando os dicionários transversais de Siglas e Abreviaturas no início do objeto JSON, servindo como um cabeçalho de referência global para todos os conceitos clínicos que se seguem.

4. Tratamento de Dados

O processo de extração e tratamento de dados foi desenvolvido de forma modular, com scripts Python dedicados a cada documento, devido às particularidades tipográficas e estruturais de cada ficheiro PDF (e respetivo XML). As bibliotecas base utilizadas em todos os módulos foram `re` (para processamento de Expressões Regulares) e `json` (para estruturação final dos dados).

4.1. *glossario_ministerio_saude.pdf*

O glossário do Ministério da Saúde apresentou-se como um documento estruturalmente desafiante, essencialmente devido ao seu ruído tipográfico (cabeçalhos e rodapés dispersos) e ao facto de conter termos médicos que, visualmente, se sobrepunham a formatações irregulares.

Metodologia e Expressões Regulares:

A estratégia de extração foi dividida num *pipeline sequencial* composto pelas seguintes fases:

- **Limpeza Direta no XML (Remoção de Ruído de Fundo):** Antes de proceder à leitura e junção das linhas de texto, efetuou-se uma limpeza diretamente sobre as tags do XML. Esta etapa foi crucial para evitar que elementos periféricos (como números de página, cabeçalhos ou siglas de referência lateral) fossem incorporados inadvertidamente nas definições médicas. A identificação destes elementos foi feita de forma inequívoca através dos seus tamanhos de fonte específicos (`font = "15", "22", "23", "25"`).

```
1
2
3  # Remove as tags de paginas, imagens e pdf2xml que nao interessam
4  texto = re.sub(r"</?page.*?>", r"", texto)
5  texto = re.sub(r"</?pdf2xml.*?>", r"", texto)
6  texto = re.sub(r"</?image.*?>", r"", texto)
7
8  # Remove numeros de pagina (15), siglas laterais (23, 25) e letras capitulares
9  (22)
10 padrao_fontes_lixo = r'<text[^>]*font="(22|15|25|23)"[^>]*>.*?</text>\n?'
11 texto = re.sub(padrao_fontes_lixo, r"", texto)
12
```

Listing 1: Limpeza direta no Xml

- **Marcação Estrutural de Termos:** Uma vez limpo o ruído periférico e unificadas as ligaturas problemáticas, avançou-se para a deteção dos termos válidos. Observou-se que, no ficheiro XML resultante, os conceitos possuíam uma assinatura tipográfica

clara: fonte tamanho 21 e formatação a negrito (), sendo frequentemente seguidos das expressões "Categoria" ou "Ver". Utilizou-se a biblioteca re para substituir estas tags de formatação por marcadores estruturais temporários, nomeadamente a string ###TERMO###.

```
1
2
3     texto = re.sub(r'</b></text>\s*<text[^>]*>\s*<b>', ' ', texto)
4     texto = re.sub(r'<text[^>]*font="21"[^>]*><b>(.*?)</b></text>', r'###TERMO###\1', texto)
5     texto = re.sub(r'<b>(.*?)</b></text>\s*<text[^>]*>\s*(?:Categoria|Ver)', r'###TERMO###\1 <text>Categoria', texto)
6
7
```

Listing 2: Marcação de termos

Esta ancoragem permitiu que o documento pudesse, finalmente, ser purgado de todas as tags XML restantes e convertido numa única string de texto simples. Subsequentemente, o método `re.split(r'###TERMO###', texto)` fatiou o documento em blocos individuais contendo apenas o termo e a sua respetiva descrição.

```
1
2
3     texto = re.sub(r'<text[^>]*>|</text>|<i>|</i>|<b>|</b>|<fontspec[^>]*>', ' ', texto)
4     texto = re.sub(r'\s+', ' ', texto).strip()
5
6     # Correcao de Hifenizacao
7     texto = re.sub(r'([a-zA-ZA-y]) - ([a-zA-ZA-y])', r'\1\2', texto)
8
9     # 4. Cortar a parte inicial
10    partes_inicio = re.split(r'###TERMO###Abordagem medica', texto, maxsplit=1)
11    if len(partes_inicio) > 1:
12        texto = "###TERMO###Abordagem medica" + partes_inicio[1]
13
14
15    blocos = re.split(r'###TERMO###', texto)
```

Listing 3: Limpeza do resto do XML

- **Processamento Iterativo e Mapeamento de Categorias:** Cada bloco extraído foi sujeito a um ciclo de processamento. Numa primeira instância, efetuou-se o isolamento das "Referências Cruzadas" (termos direcionados pela partícula "Ver"). Para os conceitos clínicos regulares, o script isolou a designação da doença e, através de um mapeamento rigoroso com uma lista pré-definida (`categorias_oficiais`), extraiu a área temática a que o termo pertencia, isolando por fim o corpo da definição.

```

1  categorias_oficiais = [
2      "Administracao e Planejamento em Saude", "Acidentes e Violencia",
3      "Alimentacao e Nutricao", "Ambiente e Saude", "Atencao a Saude",
4      "Ciencia e Tecnologia em Saude", "Ciencias Sociais em Saude",
5      "Comunicacao em Saude", "Demografia", "Direito Sanitario",
6      "Doencas", "Drogas de uso terapeutico e social", "Drogas de Uso Terapeutico
7      e Social",
8      "Economia de saude", "Economia de Saude", "Epidemiologia",
9      "Equidade em saude e social", "Etica e bioetica",
10     "Historia da saude publica", "Medicamentos, vacinas e insumos",
11     "Medicamentos, Vacinas e Insumos", "Politicas publicas e saude",
12     "Politicas Publicas e Saude", "Promocao e Educacao em Saude",
13     "Saude animal", "Vigilancia em Saude", "Recursos humanos em saude Publica",
14 ]
15
16 # 4. Extracao Iterativa de atributos
17
18 for bloco in blocos[1:]:
19
20     bloco = bloco.strip()
21     if len(bloco) < 3: continue
22
23     # 4.1 Tratamento de Referencias Cruzadas (Cross-references)
24     match_ver = re.search(r'\s+Ver\s+', bloco, re.IGNORECASE)
25
26     if match_ver:
27         partes = re.split(re.escape(match_ver.group(0)), bloco, maxsplit=1)
28         termo = re.sub(r'Categoria\s*:\s*', '', partes[0]).strip()
29         if len(termo) > 100: continue # Filtro de ruído
30
31         dicionario_final[termo] = {
32             "categoria": ["Referencia Cruzada"],
33             "descricao": "Ver " + partes[1].strip()
34         }
35
36         continue
37
38     # 4.2 Extracao de Conceitos Regulares
39
40     pos_cat = bloco.lower().find("categoria")
41
42     if pos_cat != -1:
43         termo = bloco[:pos_cat].strip()
44
45         # Filtro dimensional para mitigar a captura de indices residuais
46         if len(termo) > 100:
47             continue
48
49         resto = bloco[pos_cat:].strip()
50
51         # Limpeza do descritor semantico "Categoria:"
52         resto = re.sub(r'^categoria\s*:\s*', '', resto, flags=re.IGNORECASE)
53
54         # Correcoes ortograficas inerentes a erros de OCR/Tipografia do PDF
55         resto = resto.replace("Demografi a", "Demografia")

```



```

56     resto = resto.replace("Economia da Saude", "Economia de Saude")
57     resto = resto.replace("Ciencias Sociais e Saude", "Ciencias Sociais em
Saude")
58
59     # Identificacao e extracao da area tematica
60     cats_encontradas = []
61
62     for cat in categorias_oficiais:
63
64         if resto.lower().startswith(cat.lower()):
65             cats_encontradas.append(cat)
66             resto = resto[len(cat):].strip()
67             break
68
69     # O que sobra e a descricao
70     descricao = resto.strip()

```

Listing 4: Extracao Iterativa de atributos

- **Tratamento de Exceções:** Devido a inconsistências na formatação original do PDF (ausência de negrito em alguns subtítulos), verificou-se que determinados termos não ativaram a âncora `###TERMO###` no Passo 2. Estes conceitos "fantasmas" acabavam por ser ingeridos pelo sistema como fazendo parte da descrição clínica do termo anterior. Para garantir a integridade total dos dados, implementou-se um algoritmo pós-processamento. Este varreu o dicionário final à procura de ocorrências onde um título e a palavra "Categoria:" se encontravam ocluídos a meio de um bloco de texto.

```

1     # Algoritmo desenhado para recuperar titulos e categorias que, devido a
2     # ausencia de formatacao a negrito no PDF, ficaram embutidos na definicao do
3     termo anterior.
4     termos_a_adicionar = {}
5
6     for termo, info in dicionario_final.items():
7         descricao = info["descricao"]
8
9         # Expressao regular para detetar: [Inicio de Frase] + "Categoria:" + [
10        Continuacao]
11        match = re.search(r'([A-Z][A-Za-zA-y\s]+)\s+Categoria\s*:\s*([A-Za-zA-y\s
12        ,]+)\s+([A-Z].*)', descricao)
13        if match:
14            novo_termo = match.group(1).strip()
15            # Remocao de lixo de cabecalho anexado ao novo termo
16            novo_termo = re.sub(r'^[A-Z]{3,}\s+', '', novo_termo).strip()
17            nova_cat = match.group(2).strip()
18            nova_def = match.group(3).strip()
19
20            # Truncamento da definicao original no ponto exato da oclusao
21            texto_antes = descricao[:match.start()].strip()
22            texto_antes = re.sub(r'\s+[A-Z]{3,}$', '', texto_antes).strip()
23            dicionario_final[termo]["descricao"] = texto_antes

```

```

21
22     # Preparacao da nova instancia resgatada
23     termos_a_adicionar[novo_termo] = {
24         "categoria": [nova_cat] if nova_cat in categorias_oficiais else [],
25         "descricao": nova_def
26     }
27
28     # Adiciona os termos ao dicionario principal
29     dicionario_final.update(termos_a_adicionar)

```

Listing 5: Pós-Processamento e recuperação de oclusões

Sempre que este padrão foi detetado pela *Regex*, o sistema isolou o novo termo, recuperou a sua categoria e definição, e procedeu ao corte dessa informação residual da descrição do termo original, garantindo a separação e inclusão de ambas as entidades clínicas no ficheiro final.

O resultado, exportado no ficheiro temporário `ministerio_saude_temp.json`, estruturou todo o conteúdo com eficácia, devolvendo o dicionário consolidado livre de anomalias textuais.

```

1  {
2  "Abordagem medica tradicional do adulto hospitalizado": {
3      "categoria": [
4          "Atencao a Saude"
5      ],
6      "descricao": "Focada em uma queixa principal e o habito medico de tentar
7      explicar todas as queixas e os sinais por um unico diagnostico, que e adequada
8      no adulto jovem - nao se aplica em relacao ao idoso."
9  },
10 "Abuso financeiro dos idosos": {
11     "categoria": [
12         "Acidentes e Violencia"
13     ],
14     "descricao": "Exploracao impropria ou ilegal e/ou uso nao consentido de
15     recursos financeiros dos idosos."
16 },

```

Listing 6: JSON obtido

4.2. glossario_neologismos.pdf

O processamento deste documento revelou uma complexidade semântica superior, uma vez que a extração não se limitava ao par "Termo/Descrição". O objetivo consistiu em mapear neologismos médicos, as suas respetivas traduções para Inglês e Espanhol, e identificar siglas embutidas no corpo das definições.

Metodologia e Expressões Regulares:

A estratégia de extração seguiu um *pipeline* de processamento em três fases principais:

- **Normalização e Limpeza de Metadados:** Diferente do documento anterior, este ficheiro apresentava uma estrutura de texto contínuo. A fase inicial focou-se na normalização de espaços e quebras de linha, bem como na conversão de entidades HTML (ex: `"`) em caracteres legíveis. Utilizaram-se regras de substituição para eliminar cabeçalhos datados e números de página que interrompiam a fluidez dos parágrafos.

```

1
2 # Substitui quebras de linha e multiplos espacos por um unico espaco
3 texto = re.sub(r'\n', ' ', texto)
4 texto = re.sub(r'\s+', ' ', texto)
5 texto = re.sub(r'&#34;', '"', texto) # Transforma aspas HTML em aspas normais
6
7 # Remove delimitadores de pagina e de imagens
8 texto = re.sub(r'<page[^>]*>|</page>|<image[^>]*>', ' ', texto)
9 # Remove cabecalhos que contem o mes, ano e numeros de pagina
10 texto = re.sub(r'<text[^>]*>\s*([A-Z][a-z]+, [A-Z][a-z]+ \d{4}|\d{2,3})\s*</
11 text>', ' ', texto)

```

Listing 7: Normalização inicial de texto

- **Marcação por Âncoras Morfológicas:** Identificou-se um padrão visual constante no documento: todos os neologismos eram imediatamente seguidos pela sua classe gramatical em itálico (`<i>s.m.</i>` ou `<i>s.f.</i>`). Esta característica foi utilizada como a "âncora" principal do algoritmo. Através de *Regex*, estas ocorrências foram substituídas pelo marcador `###TERMO###`, preservando a classe gramatical entre símbolos `@` para uso posterior no split.

```

1
2 # Utiliza a tag <i> contendo a classe gramatical (s.m. ou s.f.) como ancora
3 # para isolar o neologismo e a sua respectiva classe.
4 texto = re.sub(r'<text[^>]*>\s*([<|+?])\s*</text>\s*<text[^>]*>\s*<i>\s*(s.m
5 \.|s.f.)\s*</i>\s*</text>', r'###TERMO###\1 @\2@', texto)

```

Listing 8: Identificação e Marcação dos Termos Principais

Esta técnica permitiu que, após a eliminação total das restantes tags XML, o texto pudesse ser segmentado com precisão absoluta, independentemente de quão extensa fosse a definição de cada conceito.

- **Extração Multilingue e Processamento de Blocos:** No ciclo iterativo, cada bloco de texto foi decomposto nos seus atributos fundamentais. Utilizaram-se delimitadores específicos presentes no texto original, como as chaves de idioma `[ing]` e `[esp]`, para isolar as traduções.

```

1
2   blocos = re.split(r'###TERMO###', texto)
3
4   dicionario_final = {}
5   siglas_embutidas = {}
6
7   for bloco in blocos[1:]:
8       bloco = bloco.strip()
9       if len(bloco) < 3:
10           continue
11
12       # Separa o bloco nas tres componentes primarias: [0] Termo, [1] Classe
Gramatical, [2] Restante
13       elems_gramatica = re.split(r'@(s\.m\.|s\.f\.)@', bloco, maxsplit=1)
14
15       if len(elems_gramatica) > 2:
16           termo = elems_gramatica[0].strip()
17           genero = elems_gramatica[1].strip()
18           resto = elems_gramatica[2].strip()
19
20       # 6.1. Extracao da Traducaao para o idioma Ingles
21       elems_ing = re.split(r'\[ing\];?\s*', resto, maxsplit=1)
22       trad_ing = ""
23       if len(elems_ing) > 1:
24           trad_ing = elems_ing[0].strip()
25           resto = elems_ing[1].strip()
26
27       # 6.2. Extracao da Traducaao para o idioma Espanhol
28       elems_esp = re.split(r'\[esp\]\s*|\[es\s', resto, maxsplit=1)
29       trad_esp = ""
30       if len(elems_esp) > 1:
31           trad_esp = elems_esp[0].strip()
32           resto = elems_esp[1].strip()
33

```

Listing 9: Segmentação e Processamento Iterativo dos Neologismos

Para a obtenção da descrição limpa, o algoritmo aplicou um critério de corte baseado na presença de aspas ou reticências (indicadores de citações de exemplos de uso), garantindo que apenas a definição teórica fosse capturada para o campo "descricao".

```

1
2
3   # O delimitador baseia-se na presença de aspas seguidas ou precedidas de
pontuacao
4   elems_citacao = re.split(r'\s*[""]\s*[\. \. .]+|\s*[\. \. .]+\s*[""]',
resto, maxsplit=1)
5   descricao_completa = elems_citacao[0].strip()
6
7   # Remocao de numeracoes de pagina residuais no final da string
8   descricao_completa = re.sub(r'\s*(\d+.*?)$', '', descricao_completa)
9   descricao_completa = re.sub(r'\s\d{2,3}$', '', descricao_completa)
10
11

```

Listing 10: Extração da Definição (Isolando citações e exemplos práticos)

Extração Paralela de Siglas (Desacoplamento): Um aspeto relevante deste módulo foi a capacidade de identificar o padrão **Sigla: XYZ** no início de algumas definições. O script foi desenhado para, simultaneamente, remover esta informação da descrição do conceito e guardá-la num dicionário secundário. Esta abordagem de desacoplamento permite manter o JSON de neologismos focado puramente em conceitos, enquanto alimenta de forma automatizada a base de dados transversal de siglas, que será detalhada na secção

```
1      # Captura a sigla apenas se a definicao iniciar pelo padrao "Sigla: XYZ"
2      sigla = ""
3      match_sigla = re.match(r'^Sigla:\s*([A-Z0-9\-\_])\s+(.*)',
4      descricao_completa)
5      if match_sigla:
6          sigla = match_sigla.group(1)
7          descricao_completa = match_sigla.group(2)
```

Listing 11: Extração de Sigla

O resultado final foi a geração de dois ficheiros estruturados: *neologismos_temp.json*, contendo cerca de 300 termos com traduções, e um ficheiro de siglas recuperadas, ambos devidamente ordenados e validados.

```
1      {
2      "abeta": {
3          "descricao": "Proteina que pode ser encontrada em todos os tipos de celulas do
4          organismo humano. Ao acumular-se excessivamente no cortex cerebral do ser humano pode
5          contribuir para o aceleramento do mal de alzheimer",
6          "traducoes": {
7              "EN": "abeta",
8              "ES": "abeta"
9          }
10     },
11     "acao vasoconstritora": {
12         "descricao": "Reducao do diametro das veias arterias do organismo humano, o que
13         implica na elevacao da pressao sanguinea",
14         "traducoes": {
15             "EN": "vasoconstriction",
16             "ES": "accion vasoconstritora"
17         }
18     },
19     }
```

Listing 12: JSON de conceitos obtido

4.3. wipopearl.pdf

O documento *WIPO_Pearl_COVID-19_Glossary* apresentou-se como um glossário multilíngue altamente estruturado do ponto de vista tipográfico. A principal complexidade deste ficheiro não residia no ruído textual, mas sim na necessidade de mapear traduções em nove idiomas distintos e na presença de marcadores de sinónimos (*syn.*) que surgiam tanto no conceito base (inglês) como no interior das próprias traduções.

Metodologia e Expressões Regulares:

Para o processamento deste documento, optou-se por uma estratégia de "Marcações Táticas por Fonte" (*Tactical Font Tagging*), capitalizando a extrema consistência dos tamanhos e estilos de letra do PDF original.

- **Corte das Extremidades do Documento:** Por forma a evitar o processamento de índices e páginas de metadados, o documento foi fatiado cirurgicamente utilizando as secções "Multilingual Glossary" e o rodapé final da Organização Mundial da Propriedade Intelectual como delimitadores absolutos.

```
1 # Cortar tudo o que está para trás do título do glossário
2 start_match = re.search(r'<b>Multilingual Glossary </b>', texto)
3 if start_match:
4     texto = texto[start_match.end():]
5
6 # Cortar o rodapé final comum da última página para excluir metadados
7 end_match = re.search(r'World Intellectual Property Organization', texto)
8 if end_match:
9     texto = texto[:end_match.start()]
10
```

Listing 13: Isolamento do bloco de dados útil (WIPO)

- **Marcações Táticas por Fonte (Fase 1):** Em vez de depender do posicionamento relativo do texto, o algoritmo utilizou Expressões Regulares para intercetar atributos `font` específicos do XML. Identificou-se que o conceito principal residia sempre na `font="8"` a negrito, a categoria na `font="11"`, a descrição na `font="6"` e as siglas dos idiomas na `font="7"`. Injetaram-se marcadores como `@@CONCEITO:`, `@@CATEGORIA:` e `@@LANG:` diretamente nestas ocorrências, permitindo achatar o documento e limpar o XML de forma segura.

```
1 # 1. MARCAR O CONCEITO: Sabemos pelo PDF que os conceitos base estão sempre na font
   8 a negrito
2 texto = re.sub(r'<text[^>]*font="8"[^>]*>\s*<b>\s*(.*?)\s*</b>\s*</text>', r'
   @@CONCEITO:\1@@', texto)
3
```

```

4 # 2. MARCAR A CATEGORIA: As categorias estão sempre na font 11.
5 texto = re.sub(r'<text[^>]*font="11"[^>]*>\s*(.*?)\s*</text>', r'@@CATEGORIA:\1@@',
6     texto)
7 # 3. MARCAR DESCRIÇÃO: O texto da descrição usa a font 6.
8 # Como as descrições ocupam várias linhas, marcamos cada linha individualmente. Na
9 # Fase 2 vamos colá-las.
10 texto = re.sub(r'<text[^>]*font="6"[^>]*>\s*(.*?)\s*</text>', r'@@DESC:\1@@', texto
11 )

```

Listing 14: Marcação Estrutural Baseada na Tipografia

- **Isolamento Cirúrgico de Sinónimos e Idiomas:** Um dos maiores desafios deste ficheiro consistia no facto de a string (*syn.*) pertencer tanto ao conceito em inglês como a várias traduções estrangeiras. Se a extração fosse global, os sinónimos corromperiam as traduções. Para contornar esta limitação, desenvolveu-se uma lógica de isolamento de cabeçalho: o script identificou o espaço exato entre o conceito e o início da descrição (o *header*) e aplicou a pesquisa de sinónimos exclusivamente nessa zona. Paralelamente, utilizou-se a função `re.split` nas tags de idioma para capturar o texto traduzido de forma intacta.

```

1 # 4. Extrair Sinónimos Principais
2 # Para não extrair os (syn.) das traduções, isolamos apenas a string que existe
3 # entre o conceito e o início da Descrição ou da Categoria (o "cabeçalho").
4 desc_pos = bloco_restante.find("@@DESC:")
5 cat_pos = bloco_restante.find("@@CATEGORIA:")
6
7 limites = [pos for pos in [desc_pos, cat_pos] if pos != -1]
8 limite_header = min(limites) if limites else len(bloco_restante)
9
10 # "header" contém apenas o espaço onde mora o sinónimo inglês
11 header = bloco_restante[:limite_header]
12
13 # Procuramos literalmente pela palavra (syn.) apenas no cabeçalho
14 sin_match = re.search(r'\(syn\.\)\s*(.*)', header)
15 if sin_match:
16     sin_raw = sin_match.group(1).strip()
17     sinonimos = [s.strip() for s in sin_raw.split(',') if s.strip()]
18 else:
19     sinonimos = "Sinónimos não identificados"

```

Listing 15: Isolamento do Cabeçalho e Extração de Sinónimos

O JSON resultante preservou a riqueza linguística do documento sem poluição cruzada entre as traduções.

```

1 "acute respiratory distress syndrome": {
2   "categoria": [
3     "MEDI",
4     "Pathology"
5   ],
6   "sinonimos": [
7     "ARDS"
8   ],
9   "descricao": "Respiratory disease characterized by the rapid onset of widespread
10  inflammation in the lungs.",
11   "traducoes": {
12     "DE": "akutes Atemnotsyndrom des Erwachsenen, (syn.) ARDS",
13     "ES": "síndrome de dificultad respiratoria aguda, (syn.) SDRA",
14     "FR": "syndrome de détresse respiratoire aigue, (syn.) SDRA",
15     "PT": "síndrome do desconforto respiratório agudo, (syn.) SDRA"
16   }
17 }

```

Listing 16: Exemplo de Output do wipopearl.json

4.4. medicina.pdf

O dicionário *Vocabulario de Medicina* (Galego) apresentou desafios de extração altamente irregulares. As dificuldades centraram-se na quebra inusitada de linhas nos conceitos, na variabilidade da colocação dos números de índice e no facto de siglas tipográficas imitarem visualmente as categorias médicas.

Metodologia e Expressões Regulares:

A extração foi desenhada para ser altamente resiliente a falsos positivos e a quebras de formatação da conversão do PDF.

- **Unificação de Conceitos Multilinha e Normalização de Índices:** Muitos termos médicos longos encontravam-se partidos em três ou mais linhas de texto, originando a quebra sistemática das tags . Desenvolveu-se uma rotina para "colar" estas secções, verificando se a linha seguinte abria uma nova tag não numérica. De seguida, criaram-se duas arquiteturas de captura (Caso A e Caso B) para identificar o número de índice do termo, estivesse ele dentro ou fora da formatação a negrito, eliminando posteriormente classificadores gramaticais intrusivos (como *m*, *f*, *pl*).

```

1 # Unir conceitos partidos: se um <b> fecha e o seguinte abre sem número, colamos.
2 texto = re.sub(r'</b>\s*</text>\s*<text [^>]*>\s*<b>\s*(?!\\d+\\s+)', ' ', texto)
3
4 # CASO A: O número está DENTRO da tag <b>
5 texto = re.sub(r'<text [^>]*>\s*<b>\s*(\\d+\\s+(.*?))\s*</b>\s*</text>', r'@@CONCEITO
    :\\1\\2@@', texto)

```



```

6
7 # CASO B: O número ficou numa tag <text> separada imediatamente antes do <b>
8 texto = re.sub(r'<text[^>]*>\s*(\d+)\s*</text>\s*<text[^>]*>\s*<b>\s*(.*?)\s*</b>\s*
9         *</text>', r'@@CONCEITO:\1|\2@@', texto)

```

Listing 17: Colagem de linhas e captura de índice dupla

- **O Paradoxo Tipográfico das Categorias (Font 22):** Durante a análise, detetou-se que a conversão XML colocou algumas categorias na `font="22"` em itálico. Contudo, as siglas e os marcadores de tradução também partilhavam este estilo. Se a extração fosse cega, siglas como "ADN" seriam classificadas como categorias. A solução passou por impor uma regra morfológica estrita baseada em Capitalização: para ser considerada uma categoria legítima na `font="22"`, a *Regex* exigiu que a primeira letra fosse maiúscula, seguida obrigatoriamente por letras minúsculas (rejeitando assim acrónimos integralmente em maiúsculas).

```

1 # 2. MARCAR A CATEGORIA:
2 # A maioria está na font="21".
3 texto = re.sub(r'<text[^>]*font="21"[^>]*>\s*<i>\s*(.*?)\s*</i>\s*</text>', r'
4         @@CATEGORIA:\1@@', texto)
5 # Regra font=22: Exige uma Maiúscula seguida obrigatoriamente de uma minúscula,
6   porque as traduções estão na mesma font, mas começam sempre com minúscula com
7   exceção das siglas, e assim ignoramo-las.
8 texto = re.sub(r'<text[^>]*font="22"[^>]*>\s*<i>\s*([A-ZÁÉÍÓÚŊÇ][a-záéíóúñç]+.*?)\s*
9         *</i>\s*</text>', r'@@CATEGORIA:\1@@', texto)

```

Listing 18: Regra de Validação de Categoria por Capitalização

- **Filtro Anti-Lixo e Lógica de Defesa:** Como a captura dos termos foi altamente abrangente, alguns números de página soltos do PDF foram ingeridos pelo algoritmo (assumidos como índices de conceitos válidos). Implementou-se um filtro de Programação Defensiva na Fase 2: se um bloco intercetado não possuísse simultaneamente nem categoria nem traduções, era classificado com 100% de certeza como lixo de formatação, sendo liminarmente descartado.

```

1 # Se não tem categoria nem traduções, é garantidamente um número de página perdido.
2 if categorias == "Categoria não identificada" and traducoes == "Traduções não
3   identificadas":
4   continue

```

Listing 19: Filtro de Descarte de Falsos Positivos

Esta arquitetura revelou-se extremamente fiável, estabilizando e uniformizando 5372 conceitos complexos, como ilustrado no exemplo abaixo.

```
1 "ácido gamma-aminobutírico": {  
2   "categoria": [  
3     "Bioquímica",  
4     "Fisiologia"  
5   ],  
6   "sinonimos": [  
7     "GABA (sg)"  
8   ],  
9   "variantes": "Variantes não identificadas",  
10  "descricao": "Descrição não identificada",  
11  "traducoes": {  
12    "ES": "ácido gamma-aminobutírico; GABA (sg)",  
13    "EN": "GABA (sg); gamma-aminobutyric acid",  
14    "PT": "ácido gama-aminobutírico; GABA (sg)"  
15  }  
16 }
```

Listing 20: Exemplo de Output do medicina.json

4.5. Extração e Unificação Transversal de Siglas e Abreviaturas

No decurso do processamento dos quatro glossários, constatou-se a presença de uma vasta quantidade de siglas e abreviaturas dispersas. Sob a ótica de Modelação de Dados, estabeleceu-se como requisito a separação destas entidades em dois dicionários independentes que funcionam como o preâmbulo do ficheiro consolidado. Esta estrutura permite que o utilizador ou sistema consulte o significado de uma forma abreviada antes de passar à análise detalhada dos conceitos.

O mapeamento das fontes revelou dois formatos de origem:

- **Secções Introdutórias:** Listas formais presentes no *WIPO Pearl*, *Medicina*, *Ministério da Saúde* e *Neologismos*.
- **Siglas Embutidas:** Termos extraídos do corpo das definições de *Neologismos* (ex: Sigla: AVCI), que foram resgatados e movidos para a tabela global.

Para a construção desta secção do glossário, o desenvolvimento seguiu três eixos:

- **Extração Padronizada de Secções Introdutórias:**

A regra geral identificada transversalmente nos quatro documentos foi a alocação das siglas e abreviaturas em secções de índice ou capítulos introdutórios. A metodologia adotada para a sua extração consistiu no isolamento espacial de páginas. Para evitar que os scripts de leitura capturassem ruído noutras partes dos PDFs (falsos positivos), os algoritmos foram programados para fatiar o documento e interagir exclusivamente com as coordenadas ou números de página alvo. Após

o isolamento do bloco de texto, aplicaram-se Expressões Regulares para capturar o padrão "Chave-Significado". Para automatizar a separação dos dados extraídos nestas listas, implementou-se um classificador morfológico: chaves que contivessem pontuação (ex: *et al.*) ou fossem compostas unicamente por caracteres minúsculos (ex: *pop.*) foram mapeadas para ficheiros temporários de Abreviaturas, enquanto as restantes, em maiúsculas, transitaram para ficheiros temporários de Siglas.

- **Resgate de Entidades Embutidas:**

Embora a abordagem por índices tenha sido a regra geral, o glossário de Neologismos apresentou um desafio estrutural paralelo: possuía siglas embutidas diretamente no corpo das definições clínicas (ex: **Sigla:** **AVCI**). Para garantir a padronização do JSON final, aplicou-se o Princípio da Responsabilidade Única (*Single Responsibility Principle*). O script extrator de neologismos foi adaptado para atuar como um filtro ativo: sempre que detetava o padrão identificador de uma sigla a meio de um conceito, o algoritmo capturava-a, removia-a da descrição para limpar o texto, e guardava-a de forma síncrona num ficheiro satélite (`siglas_embutidas_neologismos.json`). Este desacoplamento assegurou que nenhuma informação se perdesse e que o conceito mantivesse uma estrutura limpa.

- **Agregação e Resolução de Colisões:**

Com os ficheiros temporários de siglas e abreviaturas gerados a partir de todas as fontes, quer pelas listas de índice, quer pelo resgate de embutidos, a última fase consistiu na fusão dos dados. Para o efeito, desenvolveram-se scripts unificadores modulares capazes de iterar sobre uma lista dinâmica de ficheiros JSON. O principal desafio técnico solucionado nesta etapa foi a resolução de colisões (chaves repetidas). Uma vez que os quatro glossários partilhavam léxico médico, alguns termos foram capturados por múltiplos extratores. O algoritmo unificador foi desenhado com uma regra anti-duplicação: antes de integrar um par "Chave-Valor" no dicionário global, verificava a preexistência dessa chave, descartando redundâncias e preservando a integridade da primeira definição encontrada.

No término do processamento, ambos os dicionários foram reordenados alfabeticamente e instanciados como os blocos inaugurais do JSON final, garantindo uma arquitetura de dados otimizada e coerente.

```

1  {
2    "Siglas": {
3      "AB": "Atencao Basica",
4      "ABEn": "Associacao Brasileira de Enfermagem",
5      "ADT": "Assistencia Domiciliar Terapeutica",
6      "AFE": "Autorizacao de Funcionamento de Empresa",
7      "AIDPI": "Atencao Integrada as Doencas Prevalentes na Infancia",
8      "AIDS": "Sindrome da Imunodeficiencia Adquirida",
9      "AIH": "Autorizacao de Internacao Hospitalar",
10     ...
11
12  {
13    "Abreviaturas": {
14      "(en-US)": "English, United States",
15      "(es-AR)": "Spanish, Argentina",
16      "(es-CU)": "Spanish, Cuba",
17      "(pt-BR)": "Portuguese, Brazil",
18      "(pt-PT)": "Portuguese, Portugal",
19      "(syn.)": "synonym",
20      "a": "adjectivo",
21      "abrev": "abreviatura",
22      "AR": "Arabic",
23     ...

```

Listing 21: Jsons obtidos para as siglas e abreviaturas

5. Modelação do Dicionário Final (JSON)

A etapa final da arquitetura deste projeto consistiu na consolidação de todos os dados extraídos. Tendo em conta que o sistema gerou quatro ficheiros de conceitos independentes e dois ficheiros transversais (Siglas e Abreviaturas), o grande desafio metodológico centrou-se na fusão destas fontes de forma coesa, não destrutiva e padronizada.

Para o efeito, desenvolveu-se um script agregador mestre, cuja lógica de processamento assentou em três pilares fundamentais de modelação de dados:

Normalização Estrutural

Como os diferentes glossários apresentavam naturezas distintas (ex: o *WIPO Pearl* possuía traduções e sinónimos, enquanto o Ministério da Saúde possuía apenas categorias e descrições), a fusão direta geraria um JSON com chaves assimétricas. Para prevenir falhas em futuras consultas computacionais, implementou-se uma função de normalização estrutural (`normalizar_conceito`). Esta função atua como um molde rigoroso: se um conceito importado não possuir determinada chave, o sistema cria-a automaticamente e preenche-a com um valor nulo padronizado (ex: "Traduções não identificadas"). Isto garante que 100% das entradas no dicionário final partilhem a mesma estrutura.

Tratamento Inteligente de Colisões e Homonímia

O cruzamento de milhares de termos médicos provenientes de quatro fontes distintas resultou em inevitáveis colisões de chaves (o mesmo termo extraído de PDFs diferentes). O algoritmo (adicionar_ao_dicionario) foi desenhado para resolver estes conflitos de forma inteligente:

- **Fusão por Match Perfeito:** Se o termo e os seus metadados (categoria, sinónimos, traduções) fossem idênticos, mas as descrições divergissem, o sistema concatenava as descrições (ex: (1) Descrição A (2) Descrição B), fundindo o conhecimento sem perder dados.
- **Resolução de Homonímia:** Se o mesmo termo surgisse em contextos ou línguas totalmente diferentes (metadados divergentes), o algoritmo reconhecia a homonímia e desdobrava a chave original em instâncias numeradas independentes (ex: (1) Conceito, (2) Conceito), prevenindo a sobreposição e corrupção de dados.

Hierarquia da Árvore JSON

A modelação final do ficheiro foi desenhada para privilegiar a lógica de consulta. O dicionário foi encapsulado sob a raiz principal "Glossário de Medicina e Saúde". No seu interior, instanciaram-se três nós primários: as "Siglas" e "Abreviaturas" foram posicionadas no topo da hierarquia, atuando como *Lookup Tables globais* de rápida referência, seguidas do vasto nó de "Conceitos", onde toda a terminologia clínica unificada foi alojada.

Abaixo apresenta-se uma representação visual da estrutura do ficheiro `glossario_final.json`:

```
1  {
2    "Glossario de Medicina e Saude": {
3      "Siglas": {
4        "AB": "Atencao Basica",
5        "ABEn": "Associacao Brasileira de Enfermagem",
6        "ADT": "Assistencia Domiciliar Terapeutica",
7        "AFE": "Autorizacao de Funcionamento de Empresa",
8        "AIDPI": "Atencao Integrada as Doencas Prevalentes na Infancia",
9        "AIDS": "Sindrome da Imunodeficiencia Adquirida",
10       "AIH": "Autorizacao de Internacao Hospitalar",
11       "AIS": "Acoes Integradas de Saude",
12       ...
13     "Abreviaturas": {
14       "(en-US)": "English, United States",
15       "(es-AR)": "Spanish, Argentina",
16       "(es-CU)": "Spanish, Cuba",
17       "(pt-BR)": "Portuguese, Brazil",
18       "(pt-PT)": "Portuguese, Portugal",
19       "(syn.)": "synonym",
```

```

20     "a": "adjectivo",
21     "abrev": "abreviatura",
22     "AR": "Arabic",
23     ...
24     "Conceitos": {
25         "a": {
26             "categoria": [
27                 "Anatomia"
28             ],
29             "sinonimos": "Sinonimos nao identificados",
30             "variantes": "Variantes nao identificadas",
31             "descricao": "Descricao nao identificada",
32             "traducoes": {
33                 "ES": "ala",
34                 "EN": "wing",
35                 "PT": "asa",
36                 "LA": "ala"
37             }
38         },
39         "a termo": {
40             "categoria": [
41                 "Fisioloxia"
42             ],
43             "sinonimos": "Sinonimos nao identificados",
44             "variantes": "Variantes nao identificadas",
45             "descricao": "Descricao nao identificada",
46             "traducoes": {
47                 "ES": "a termino",
48                 "EN": "full term",
49                 "PT": "a termo"
50             }
51         }

```

6. Desafios e Limitações Encontradas

O desenvolvimento deste projeto evidenciou a complexidade inerente ao Processamento de Linguagem Natural quando aplicado a documentos não nativamente digitais ou provenientes de conversões imperfeitas. A abordagem baseada em regras (*Rule-based Parsing*), através do uso extensivo de Expressões Regulares (*Regex*), revelou-se altamente eficaz na extração de dados, mas também expôs as suas próprias limitações e os desafios intrínsecos à heterogeneidade dos ficheiros originais. Destacam-se os seguintes aspetos críticos:

A Qualidade e Inconsistência da Conversão PDF-XML

A limitação mais transversal a todo o projeto prendeu-se com a qualidade e instabilidade da conversão dos ficheiros .pdf para .xml. Fatores como a hifenização de quebra de linha (que fragmentou palavras a meio, ex: pa- lavra) e a geração de ligaduras tipográficas (onde caracteres como "fi" e "fl" foram fundidos pelo conversor em símbolos Unicode únicos) obrigaram à criação de camadas densas de pré-processamento para higienização dos dados.

A isto somou-se a fragmentação estrutural de conceitos. Particularmente no *Voca-*

bulário de Medicina, termos clínicos mais longos que transitavam para a linha seguinte originavam a quebra artificial e repetida das tags de formatação (ex: `<b>`), exigindo o desenvolvimento de rotinas computacionais para a "colagem"semântica dessas quebras. Adicionalmente, a ausência de uma estruturação HTML pura no PDF forçou o algoritmo a depender de metadados visuais frágeis (como os atributos `font="21"` ou `left="174"`). Esta dependência revelou-se um desafio quando o conversor apresentava comportamentos imprevisíveis, fundindo esporadicamente chaves e descrições na mesma etiqueta de texto (*tag*), ou separando-as em blocos distintos, o que obrigou à implementação de máquinas de estados nos scripts para lidar com a assimetria das leituras.

Anomalias Tipográficas, Ambiguidade e Oclusão de Dados

Durante a fase de extração, a equipa deparou-se com falhas severas na formatação original dos documentos. Em determinados casos, a ausência de marcação visual (como o destaque a negrito no título de um termo) provocou a oclusão de dados. Estes "termos fantasma" não acionavam as âncoras de segmentação baseadas em *Regex*, acabando por ser indevidamente ingeridos pelo sistema como fazendo parte da descrição clínica do termo anterior, o que exigiu algoritmos de resgate de dados em pós-processamento.

Por outro lado, o excesso de consistência tipográfica gerou o problema oposto: a ambiguidade de metadados. No dicionário de Medicina, siglas puras (ex: **ADN**, **GABA**) partilhavam a exata mesma fonte e estilo (`font="22"` em itálico) que algumas categorias médicas, sendo que a fonte e estilo comum destas últimas era `font="21"` em itálico. A extração puramente visual provocou inicialmente o alojamento destas siglas nas listas de categorias. A resolução deste impasse obrigou a uma evolução da arquitetura do código, transitando de uma análise meramente tipográfica para uma análise morfológica (impondo regras estritas de capitalização em *Regex* para distinguir siglas de nomes comuns).

Por fim, verificou-se o desafio da contaminação multicontextual de marcadores. No glossário multilingue (*WIPO Pearl*), marcadores textuais como (*syn.*) não eram exclusivos da língua inglesa, surgindo embutidos no interior das traduções de vários outros idiomas. Uma captura global destes marcadores corromperia a integridade do mapeamento multilingue. Para colmatar esta limitação de contexto inerente às Expressões Regulares, foi necessário desenvolver lógicas de "isolamento cirúrgico", fatiando o bloco de texto para garantir que a pesquisa por sinónimos ocorresse exclusivamente no cabeçalho de cada conceito. Adicionalmente, a elevada abrangência necessária nas regras de captura atraiu esporadicamente lixo residual (como números de página solitários), o que foi mitigado com sucesso através da aplicação de heurísticas de programação defensiva e

"Filtros Anti-Lixo".

7. Conclusão

O presente trabalho prático demonstrou a exequibilidade e a relevância da aplicação de técnicas de Processamento de Linguagem Natural (PLN) na estruturação de dados não formatados. Através do uso estratégico de Expressões Regulares em Python, foi possível processar quatro glossários médicos distintos, caracterizados por uma elevada assimetria estrutural e ruído tipográfico, unificando-os num dicionário JSON coeso e pesquisável.

O desenvolvimento modular adotado, com scripts dedicados para cada fonte e um *pipeline* final de fusão (*merge*) com verificação de duplicados, revelou-se uma decisão arquitetural robusta. Esta metodologia permitiu não só a extração limpa de conceitos clínicos, descrições e traduções multilingues, mas também a criação de dicionários transversais de Siglas e Abreviaturas, posicionados no preâmbulo do ficheiro final para consulta otimizada.

Apesar dos desafios inerentes à qualidade de conversão do formato PDF e à rigidez típica do *Rule-based Parsing*, que exigiram o desenvolvimento de algoritmos de pós-processamento e recuperação de oclusões, o objetivo central de consolidação terminológica foi cumprido com elevado grau de eficácia.